DEVELOPING CODE · DOMAIN 1 - DEVELOPING CODE FOR DATA PROCESSING

AUTO CDC and slowly changing dimensions

Section 1. Out-of-order changes, SCD Type 1 and Type 2, and why MERGE is not enough.

SECTION 1 of 10 WEIGHT 22% TOPIC AUTO CDC

admin@sthithapragna.com

A merge applies changes in the order they arrive. AUTO CDC applies them in the order they happened.

AUTO CDC and APPLY CHANGES are the same thing

- **AUTO CDC replaces APPLY CHANGES, with identical syntax.** Databricks recommends the newer name.
- APPLY CHANGES still works. Both names appear across documentation, courses and question banks.
- In Python: **create_auto_cdc_flow** replaced **apply_changes**.
- **Not supported by Apache Spark Declarative Pipelines.** This is a Databricks extension, not open source.

> ON THE EXAM
>
> The July 2026 exam guide writes it as 'AUTO CDC APIs (formerly APPLY CHANGES)', so expect either name in a question.

Which one depends on the source

API	USE IT WHEN
AUTO CDC	The source has a change feed. You receive inserts, updates and deletes as events.
AUTO CDC FROM SNAPSHOT	The source has no change feed. You receive whole-table snapshots.

ON THE EXAM

AUTO CDC works in both SQL and Python. AUTO CDC FROM SNAPSHOT is Python only, and the SQL interface does not support it.

What has to be true before you can use it

- The pipeline must be on **serverless**, or on the **Pro or Advanced** pipeline editions.
- The Core edition will not run it. This is an edition-level gate, not a runtime version.
- You must **declare the target streaming table first**, then define the flow that writes into it.
- The source must be a **streaming source**. In SQL, read it with the stream keyword.

> ON THE EXAM
>
> Two-step construction: create the streaming table, then create the flow. A question showing only one step is describing something that does not run.

What each part of the statement does

CLAUSE	MEANING
KEYS	The columns that identify a record uniquely.
SEQUENCE BY	Which change happened first. This is the whole point.
APPLY AS DELETE WHEN	The condition that marks an event as a delete.
APPLY AS TRUNCATE WHEN	Clears the table at that point in the sequence.
COLUMNS * EXCEPT	Which source columns land in the target.
STORED AS	SCD TYPE 1, SCD TYPE 2, or BITEMPORAL.
TRACK HISTORY ON	Which columns cause a new version. Defaults to all of them.

ON THE EXAM

SEQUENCE BY is what makes this different from a merge.

SCD Type 1 against Type 2

SCD TYPE 1

Only the current truth

- Updates records in place
- No history is retained
- The target holds one row per key
- No start or end columns are added

SCD TYPE 2

Every version, dated

- A new row for each version
- Adds a start-at and an end-at column
- An end-at of null means currently active
- History survives deletes as a closed row

THE TIEBREAKER

Type 1 answers 'what is true now'. Type 2 answers 'what did we believe on this date'. Audit and point-in-time reporting need Type 2.

One user, three cities, two outcomes

	RESULT FOR USER 125
Input events	Tijuana at sequence 2, Mexicali at sequence 5, Guadalajara at sequence 6.
Arrival order	Guadalajara arrives before Mexicali. The feed is out of order.
SCD TYPE 1 target	One row: Guadalajara. The Mexicali update is dropped as superseded.
SCD TYPE 2 target	Three rows: Tijuana 2 to 5, Mexicali 5 to 6, Guadalajara 6 to null.

ON THE EXAM

Note what Type 1 did with the late arrival: it dropped it, because a higher sequence number had already been applied. A merge would have overwritten with the stale value.

Start at, end at, and what null means

- Type 2 adds a **start-at** and an **end-at** column, holding values from the sequencing column.
- A row with **end-at of null is the current version**. That is how you filter for present state.
- If you declare the target schema yourself, you **must include both columns**, with the same type as the sequence column.
- A deleted record does not disappear. Its final version is closed with an end value.

> ON THE EXAM
>
> Deletion in Type 2 means ending the current version, not removing rows. History of a deleted entity is preserved deliberately.

Not every change deserves a new version

- By default, **any** column change creates a new version. That is TRACK HISTORY ON everything.
- Name a subset, or exclude columns, and changes to the untracked ones **overwrite the current row in place**.
- Useful when a dimension has volatile attributes you do not want cluttering the history.
- The excluded column still exists and still updates. It just does not trigger a new row.

> ON THE EXAM
>
> Excluding a column from history tracking does not exclude it from the table. It changes when a version boundary is created.

Two conditions, both optional

- **APPLY AS DELETE WHEN** marks matching events as deletions rather than upserts.
- **APPLY AS TRUNCATE WHEN** clears the table at that point in the sequence.
- A truncate is **positional**. Records inserted before it are removed; later ones survive.
- For Type 2, deleted rows are held briefly as tombstones, with a view filtering them out.

> ON THE EXAM
>
> Truncate applies at its sequence position, not on arrival. That is why it can remove records that were inserted earlier in the sequence.

Initial hydration with a once flow

- A **once** flow performs a one-time insert, or backfill, into the target.
- It is **not re-run** when the pipeline refreshes, except during a full refresh.
- The documented pattern for replicating a database: hydrate once from a snapshot, then process the ongoing feed.
- This solves the problem that a change feed usually starts partway through a table's life.

> ON THE EXAM
>
> A change feed tells you what changed since it was enabled. It does not tell you what was already there. That gap is what a once flow fills.

Choosing the sequence column carefully

- The sequencing column must be a **sortable type**, and **null values are not supported**.
- To sequence by more than one column, combine them in a **struct**. It orders by the first field, then breaks ties on the second.
- That is how you handle a timestamp that is not unique, adding an identifier to break ties.
- Bitemporal processing adds a second sequence, for system time alongside business time.

> ON THE EXAM
>
> Nulls in the sequence column are a stated limitation, not a warning. If the source can emit them, that has to be handled upstream.

When there is no change feed

- The snapshot API compares successive snapshots to work out what changed, then applies those changes.
- Snapshots can come from a Delta table, cloud storage files, or a database connection.
- Two patterns: rely on the **pipeline run** for ordering, or supply a **version function** for explicit control.
- With a version function, a snapshot that arrives **out of order is skipped**, not reordered.

> ON THE EXAM
>
> Under Type 1 the target looks exactly like the latest snapshot. The value is that downstream consumers can read the change feed instead of the whole table.

What to carry into the exam

- AUTO CDC exists because **merge applies arrival order**, and change feeds arrive out of order.
- **SEQUENCE BY** is the clause that fixes it. Type 1 keeps current state, Type 2 keeps versions.
- Type 2 adds start and end columns. **End of null means current.**
- Requires **serverless or Pro or Advanced**. Snapshot processing is **Python only**.
- A **once** flow handles initial hydration before the change feed takes over.

> ON THE EXAM
>
> If an option proposes hand-written merge logic for an out-of-order CDC feed, it is describing the problem this API exists to solve.

A team replicates a customer table using MERGE INTO in a scheduled job, matching on customer id and updating when matched. The source emits changes with a monotonically increasing version number.

Auditors find several customers whose current address is an older value than one recorded earlier. The merge logic has been reviewed and matches on the correct key.

What is the most likely cause?

A The merge is matching on a non-unique key, so multiple rows are being updated.

B Change events are arriving out of order, and MERGE applies them in arrival order.

C The target table needs change data feed enabled to preserve update ordering.

D The scheduled job interval is too long, so updates are being batched together.

◆ PAUSE AND DECIDE

What is the most likely cause?

A The merge is matching on a non-unique key, so multiple rows are being updated.

The question states the key is correct, and this would produce different symptoms.

B Change events are arriving out of order, and MERGE applies them in arrival order.

The documented failure mode. A version number exists in the source and the merge is ignoring it.

C The target table needs change data feed enabled to preserve update ordering.

Change data feed records changes to a Delta table. It does not order incoming events.

D The scheduled job interval is too long, so updates are being batched together.

Batching is not the problem. Within a batch the order is still arrival order.

ON THE EXAM

The giveaway is that a version number exists and the merge does not use it. That column is exactly what SEQUENCE BY consumes.

> A product dimension must support point-in-time reporting: analysts need to reconstruct what each product's category and price were on any past date. Products are occasionally deleted from the source.
>
> The team implements AUTO CDC with STORED AS SCD TYPE 1 and reports that deleted products vanish and old prices are unavailable.

What should change?

A Keep Type 1 and enable change data feed on the target so history can be reconstructed.

B Use SCD TYPE 2, which retains every version and closes deleted records rather than removing them.

C Keep Type 1 and remove the APPLY AS DELETE WHEN clause so deletions are ignored.

D Use AUTO CDC FROM SNAPSHOT, which retains history automatically.

◆ PAUSE AND DECIDE

What should change?

A Keep Type 1 and enable change data feed on the target so history can be reconstructed.

Reconstructing a dimension from a change feed is exactly the manual work Type 2 removes.

B Use SCD TYPE 2, which retains every version and closes deleted records rather than removing them.

Point-in-time reporting is the definition of Type 2, and deletions close a version rather than dropping the row.

C Keep Type 1 and remove the APPLY AS DELETE WHEN clause so deletions are ignored.

Products would linger as though still active, which is a different kind of wrong.

D Use AUTO CDC FROM SNAPSHOT, which retains history automatically.

The snapshot API also offers Type 1 or Type 2. The source API is not what governs history.

ON THE EXAM

D is worth noticing: it swaps the wrong axis. Snapshot versus feed is about the source, not about whether history is kept.

> A supplier dimension uses SCD Type 2. It has a last_contacted timestamp that updates several times a day, and a credit_rating that changes twice a year.
>
> Analysts complain that the dimension has grown enormous and that finding credit rating changes means wading through thousands of versions per supplier.

What is the appropriate fix?

A Switch the dimension to SCD Type 1 and store credit rating history in a separate table.

B Use TRACK HISTORY ON with credit_rating, so only that column creates a new version.

C Exclude last_contacted with the COLUMNS clause so it is not written to the target.

D Add last_contacted to the KEYS clause so versions are grouped by supplier and time.

◆ PAUSE AND DECIDE

What is the appropriate fix?

A. Switch to SCD Type 1 and store credit rating history in a separate table.

Rebuilds by hand the history mechanism already available, and abandons Type 2 for the whole dimension.

B. Use TRACK HISTORY ON with credit_rating, so only that column creates a new version.

Untracked columns update the current row in place. The timestamp stays available and stops creating versions.

C. Exclude last_contacted with the COLUMNS clause so it is not written to the target.

Removes the column from the table entirely. It was wanted, just not as a version trigger.

D. Add last_contacted to the KEYS clause so versions are grouped by supplier and time.

Keys identify a record. Adding a changing column would make every event a new supplier.

ON THE EXAM

C and B differ in one word: whether the column is in the table, or whether it creates a version. Read which one the scenario actually wants.

A team must replicate an operational database table into the lakehouse. Change data capture was enabled on the source last week, but the table has eight years of existing rows.

They configure AUTO CDC against the change feed. The target contains only the last week of activity, and rows never touched since CDC was enabled are missing entirely.

What is the correct approach?

A Run a full refresh on the target streaming table to pull in the historical rows.

B Use a once flow to hydrate from a full extract, then continue with the change feed.

C Switch to AUTO CDC FROM SNAPSHOT, which reads the whole table each run.

D Extend the change feed retention on the source database to eight years.

◆ PAUSE AND DECIDE

What is the correct approach?

A Run a full refresh on the target streaming table to pull in the historical rows.

Replays the same change feed from the beginning. The feed still only holds one week.

B Use a once flow to hydrate from a full extract, then continue with the change feed.

The documented pattern for replicating an existing database. The once flow fills the gap the feed cannot.

C Switch to AUTO CDC FROM SNAPSHOT, which reads the whole table each run.

Would work, and discards a real change feed to repeatedly copy an eight-year table.

D Extend the change feed retention on the source database to eight years.

Retention cannot create events for changes that happened before CDC was enabled.

ON THE EXAM

A change feed knows what changed since it started. Rows that never changed since then were never events, so no amount of replay produces them.

> An engineer writes an AUTO CDC flow sequencing by an event timestamp. The source system generates timestamps at second resolution and can emit several changes to the same record within one second.
>
> Downstream, some records show the wrong final value, and which one is wrong varies between pipeline runs.

What is the cause and remedy?

A. Ties in the sequence column are resolved arbitrarily. Sequence by a struct of timestamp and a tie-breaking id.

B. Second resolution is too coarse for AUTO CDC. Convert the timestamp to milliseconds.

C. The sequence column must be numeric. Cast the timestamp to a long.

D. Add the timestamp to the KEYS clause so each change is treated as distinct.

◆ PAUSE AND DECIDE

What is the cause and remedy?

A Ties in the sequence column are resolved arbitrarily. Sequence by a struct of timestamp and a tie-breaking id.

A struct orders by the first field and breaks ties on the second. This is the documented pattern for non-unique timestamps.

B Second resolution is too coarse for AUTO CDC. Convert the timestamp to milliseconds.

Does not add information that was never captured. The source only knows the second.

C The sequence column must be numeric. Cast the timestamp to a long.

It must be sortable, and timestamps are. Casting changes nothing about the ties.

D Add the timestamp to the KEYS clause so each change is treated as distinct.

Every change would become a separate record rather than a version of one.

ON THE EXAM

Varying between runs is the signal. Non-determinism means something is being decided arbitrarily, and ties are the usual cause.

> A team on the Core pipeline edition wants to build an SCD Type 2 dimension. Their source is a legacy system with no change data capture, delivering a nightly full extract as CSV files to cloud storage.
>
> They plan to write the logic in SQL, as nobody on the team writes Python.

What must they change before this will work?

A. Nothing. AUTO CDC FROM SNAPSHOT works in SQL on any pipeline edition.

B. Move to serverless or the Pro or Advanced edition, and accept that snapshot processing requires Python.

C. Enable change data capture on the legacy source, which AUTO CDC requires.

D. Upgrade to a newer Databricks Runtime, which unlocks the CDC APIs.

◆ PAUSE AND DECIDE

What must they change before this will work?

A Nothing. AUTO CDC FROM SNAPSHOT works in SQL on any pipeline edition.

Wrong twice: the edition gate is real, and snapshot processing has no SQL interface.

B Move to serverless or the Pro or Advanced edition, and accept that snapshot processing requires Python.

Two independent blockers, and both must be cleared. Snapshots suit the source; Python is the only interface for them.

C Enable change data capture on the legacy source, which AUTO CDC requires.

Would let them use the SQL interface, and the snapshot API exists precisely so this is not required.

D Upgrade to a newer Databricks Runtime, which unlocks the CDC APIs.

The gate is the pipeline edition, not the runtime version.

ON THE EXAM

Two blockers, not one. Scan a scenario for every stated constraint before choosing, because these questions often hide a second.